Спеціальні розділи обчислювальної математики
Комп'ютерний практикум №2

Багаторозрядна модулярна арифметика

ФІ-43 Редько Микита

4 листопада 2026 р.

1 Мета роботи

Отримання практичних навичок програмної реалізації багаторозрядної арифметики; ознайомлення з прийомами ефективної реалізації критичних по часу ділянок програмного коду та методами оцінки їх ефективності.

2 Реалізація

Бібліотеку для роботи з m-бітними цілими числами ($m = 2048$), створену на комп'ютерному практикумі №1, доповнено такими операціями:

- gcd_big_int, lcm_big_int — обчислення НСД (алгоритм Евкліда) та НСК ($\frac{a}{\gcd(a,b)} \cdot b$) двох чисел;
- barrett_precompute_big_int — одноразове обчислення $\mu = \lfloor \beta^{2k}/n \rfloor$ для модуля n, де $\beta = 2^{64}$, а k — кількість розрядів n;
- barrett_reduction_big_int — редукція Барретта $x \bmod n$ для $x < \beta^{2k}$ (якщо $x < n$, число повертається без обчислень);
- add_mod_big_int, subtract_mod_big_int — додавання та віднімання чисел за модулем (з одним умовним відніманням або додаванням n);
- mult_mod_big_int, square_mod_big_int — множення чисел та піднесення чисел до квадрата за модулем;
- horner_exponentiation_mod_big_int — піднесення числа до багаторозрядного степеня d за модулем n за схемою Горнера;
- window_exponentiation_mod_big_int — модулярне піднесення до степеня віконним методом (4 біти) з таблицею $a^0, \ldots, a^{15}$ (додаткове завдання, варіант 6).

Згідно з рекомендаціями завдання, модулярну арифметику реалізовано на базі редукції Барретта, а піднесення до степеня — на базі схеми Горнера. Усі модулярні функції приймають наперед обчислену структуру BarrettModulus, тому μ обчислюється один раз для кожного модуля. Числа мають фіксовану довжину 2048 біт (32 розряди), а проміжний добуток $q \cdot \mu$ у редукції Барретта займає до $2k + 2$ розрядів, тому модуль може мати не більше 15 розрядів (960 біт).

3 Результати

Для контролю коректності реалізації алгоритмів (пункт Б завдання) тотожності перевіряються для декількох випадкових багаторозрядних a, b, c, n. Для генерації випадкових чисел та модулів заданої довжини використано такі допоміжні функції:

```rust
fn random_big_int(limbs: usize) -> BigInt {
    let mut rng = rand::rng();
    BigInt {
        limbs: std::array::from_fn(|i| if i < limbs { rng.random() } else { 0 }),
    }
}

fn random_modulus(limbs: usize) -> BarrettModulus {
    let mut n = random_big_int(limbs);
    n.limbs[limbs - 1] |= 1u64 << 63;
    barrett_precompute_big_int(&n).unwrap()
}
```

3.1 Дистрибутивність

Перевіряється $(a+b)\cdot c \equiv c\cdot(a+b) \equiv a\cdot c + b\cdot c \pmod{n}$ для модулів довжиною від 1 до 15 розрядів.

```rust
#[test]
fn test_mod_distributivity() {
    for limbs in 1..BIG_INT_LIMBS / 2 {
        let modulus = random_modulus(limbs);
        let a = random_big_int(limbs);
        let b = random_big_int(limbs);
        let c = random_big_int(limbs);

        let sum = add_mod_big_int(&a, &b, &modulus).unwrap();
        let sum_times_c = mult_mod_big_int(&sum, &c, &modulus);
        let c_times_sum = mult_mod_big_int(&c, &sum, &modulus);
        let a_c_plus_b_c = add_mod_big_int(
            &mult_mod_big_int(&a, &c, &modulus).unwrap(),
            &mult_mod_big_int(&b, &c, &modulus).unwrap(),
            &modulus,
        );

        assert_eq!(sum_times_c, c_times_sum);
        assert_eq!(sum_times_c, a_c_plus_b_c);
    }
}
```

3.2 Множення як повторне додавання

Перевіряється $n\cdot a \equiv \underbrace{a + a + \cdots + a}_{n} \pmod{m}$, де n — випадкове число з $[100, 200]$ (не менше за 100).

```rust
#[test]
fn test_mod_mult_by_repeated_addition() {
    for limbs in 1..BIG_INT_LIMBS / 2 {
        let modulus = random_modulus(limbs);
        let a = random_big_int(limbs);
        let count = rand::rng().random_range(100..=200);

        let mut sum = u64_to_big_int(0);
        for _ in 0..count {
```

```
            sum = add_mod_big_int(&sum, &a, &modulus).unwrap();
        }

        assert_eq!(
            mult_mod_big_int(&u64_to_big_int(count), &a, &modulus),
            Some(sum)
        );
    }
}
```

3.3 Теорема Ейлера

Перевіряється $a^{\varphi(n)} \equiv 1 \pmod{n}$ (за умови $\gcd(a, n) = 1$) обома методами піднесення до степеня. Для простого n (число Мерсенна $2^{521} - 1$) $\varphi(n) = n - 1$:

```
#[test]
fn test_euler_theorem_prime() {
    let n = hex_to_big_int(PRIME_521).unwrap();
    let modulus = barrett_precompute_big_int(&n).unwrap();
    let phi = subtract_big_int(&n, &u64_to_big_int(1)).unwrap();

    for _ in 0..3 {
        let a = random_big_int(modulus.k);

        assert_eq!(gcd_big_int(&a, &n), Some(u64_to_big_int(1)));
        assert_eq!(
            horner_exponentiation_mod_big_int(&a, &phi, &modulus),
            Some(u64_to_big_int(1))
        );
        assert_eq!(
            window_exponentiation_mod_big_int(&a, &phi, &modulus),
            Some(u64_to_big_int(1))
        );
    }
}
```

Для $n = 3^k$, $k \in \{1, 40, 200, 605\}$, та довільного a, що не ділиться на 3, $\varphi(3^k) = 3^k - 3^{k-1} = 2 \cdot 3^{k-1}$, отже, має виконуватись $a^{2 \cdot 3^{k-1}} \equiv 1 \pmod{3^k}$:

```
#[test]
fn test_euler_theorem_power_of_three() {
    let three = u64_to_big_int(3);
    for k in [1, 40, 200, 605] {
        let n = horner_exponentiation_big_int(&three, &u64_to_big_int(k));
        let previous_power = horner_exponentiation_big_int(&three, &u64_to_big_int(k - 1));
        let phi = subtract_big_int(&n, &previous_power).unwrap();
        let modulus = barrett_precompute_big_int(&n).unwrap();

        for _ in 0..3 {
            let mut a = random_big_int(modulus.k);
            if divide_mod_big_int(&a, &three).unwrap().1 == u64_to_big_int(0) {
                a = add_big_int(&a, &u64_to_big_int(1));
            }

            assert_eq!(gcd_big_int(&a, &n), Some(u64_to_big_int(1)));
            assert_eq!(
                horner_exponentiation_mod_big_int(&a, &phi, &modulus),
                Some(u64_to_big_int(1))
            );
            assert_eq!(
                window_exponentiation_mod_big_int(&a, &phi, &modulus),
                Some(u64_to_big_int(1))
```

```
            );
        }
    }
}
```

3.4 Власні тести

Окрім тотожностей із завдання, перевіряються:

- результати всіх модулярних операцій для фіксованих 950-бітних чисел у порівнянні з наперед обчисленими значеннями;
- збіг редукції Барретта з остачею від ділення для випадкових $x < \beta^{2k}$;
- $(a + b) - b \equiv a \pmod{n}$;
- $\gcd(a, b) \cdot \mathrm{lcm}(a, b) = a \cdot b$ для чисел зі спільним дільником;
- збіг віконного методу з методом Горнера для показників від 0 до 299;
- граничні випадки: нульовий та задовгий модуль, задовгий аргумент редукції, НСД і НСК з нулем, нульовий показник степеня.

4 Підтвердження проходження тестів

Нижче наведено результат запуску cargo test для тестів цього практикуму; усі 62 тести бібліотеки пройшли успішно.

```
test tests::test_add_mod_big_int ... ok
test tests::test_add_mod_big_int_wrap ... ok
test tests::test_add_subtract_mod_inverse ... ok
test tests::test_barrett_precompute_invalid_modulus ... ok
test tests::test_barrett_reduction ... ok
test tests::test_barrett_reduction_division_consistency ... ok
test tests::test_barrett_reduction_random_division_consistency ... ok
test tests::test_barrett_reduction_too_long ... ok
test tests::test_euler_theorem_power_of_three ... ok
test tests::test_euler_theorem_prime ... ok
test tests::test_gcd_a_greater_or_equal_b ... ok
test tests::test_gcd_a_less_than_b ... ok
test tests::test_gcd_lcm_product ... ok
test tests::test_gcd_zero ... ok
test tests::test_horner_exponentiation_mod_big_int ... ok
test tests::test_horner_exponentiation_mod_big_int_zero_exponent ... ok
test tests::test_kill_big_int_last_limbs ... ok
test tests::test_lcm_big_int ... ok
test tests::test_lcm_zero ... ok
test tests::test_mod_distributivity ... ok
test tests::test_mod_mult_by_repeated_addition ... ok
test tests::test_mult_mod_big_int ... ok
test tests::test_square_mod_big_int ... ok
test tests::test_subtract_mod_big_int_a_greater_than_b ... ok
test tests::test_subtract_mod_big_int_a_less_than_b ... ok
test tests::test_window_exponentiation_mod_big_int ... ok
test tests::test_window_exponentiation_mod_big_int_zero_exponent ... ok
test tests::test_window_horner_exponentiation_mod_consistency ... ok

test result: ok. 62 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 1.93s
```

5 Результати бенчмарків

Середній час виконання реалізованих операцій (пункт B завдання) наведено в таблиці 1 у наносекундах на одну операцію. Модуль і показники степеня — випадкові 960-бітні числа, операнди — випадкові лишки за цим модулем (модуль один для всіх вимірювань, μ обчислено до початку вимірювань). Редукції застосовано до добутку двох операндів. Для операцій за модулем виконано 1000 прогрівальних запусків і 10000 вимірювань, для НСД і НСК — 100 і 1000, для піднесення до степеня — 10 і 100. Результати отримано командою cargo run --release.

Операція	**Час, нс/операцію**
Редукція Барретта	2769.9
Редукція діленням з остачею	33751.4
Додавання за модулем	64.4
Віднімання за модулем	70.8
Множення за модулем	4182.5
Піднесення до квадрата за модулем	3289.2
НСД	95071.5
НСК	131629.7
Піднесення до степеня за модулем методом Горнера	5092995.8
Піднесення до степеня за модулем віконним методом (4 біти)	4104750.0

Табл. 1: Середній час виконання операцій

Редукція Барретта приблизно в 12 разів швидша за редукцію діленням з остачею. Для аргументів, менших за модуль, редукція виконує лише одне порівняння, тому додавання та віднімання за модулем займають десятки наносекунд, а час множення та піднесення до квадрата за модулем складається з однієї редукції Барретта та самої операції.

Віконний метод приблизно на 19% швидший за метод Горнера. Для 960-бітного показника обидва методи виконують близько 960 піднесень до квадрата, але метод Горнера виконує множення для кожного одиничного біта (близько 480), а віконний метод — для кожного ненульового вікна (близько 225, плюс 15 множень на побудову таблиці).